

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..20..29

SETUP

 Copy as Markdown

Set up passkey sign-in

Passkeys provide a passwordless authentication method using WebAuthn/FIDO2 standards. They offer a secure, phishing-resistant way for users to sign in using biometric authentication, security keys, or device PINs.

WARNING

The passkey identity provider is **experimental**. Flutter UI components for passkeys aren't available yet, so build your own UI with the generated client endpoints.

CAUTION

You need to install the auth module before you continue, see [Setup](#).

Server-side configuration

Passkeys require minimal configuration. You will need only the hostname (relying party ID) that will be associated with your passkeys.

INFO

The hostname should match the domain where your application is hosted. For development, you can use `localhost` or your development domain.

Configure the Passkey identity provider

In your main `server.dart` file, configure the Passkey identity provider by setting the `PasskeyIdpConfig` in your `pod.initializeAuthServices()` configuration:



```
import 'package:serverpod_auth_idp_server/core.dart';
import 'package:serverpod_auth_idp_server/providers/passkey.dart';

import 'src/generated/serverpod.dart';

void run(List<String> args) async {
  final pod = Serverpod(args);

  // Configure Passkey identity provider
  pod.initializeAuthServices(
    tokenManagerBuilders: [
      JwtConfigFromPasswords(),
    ],
    identityProviderBuilders: [
      PasskeyIdpConfig(
        hostname: pod.getPassword('passkeyHostname')!,
      ),
    ],
  );

  await pod.start();
}
```

TIP

You can use the `PasskeyIdpConfigFromPasswords` constructor in replacement of the `PasskeyIdpConfig` above to automatically load the hostname from the `config/passwords.yaml` file or environment variables. It will expect the `passkeyHostname` key on the file or the `SERVERPOD_PASSWORD_passkeyHostname` environment variable.

Then, extend the abstract endpoint to expose it on the server:

```
import 'package:serverpod_auth_idp_server/providers/passkey.dart';

class PasskeyIdpEndpoint extends PasskeyIdpBaseEndpoint {}
```

Finally, start the server with `serverpod start` to generate the client code, then create and apply the migration that initializes the database for the provider (in the `serverpod start` terminal, press **M**). More detailed instructions can be found in the general [identity providers setup](#) section.

Basic configuration options

- `hostname`: Required. The hostname (relying party ID) to be used on the web and associated with any apps. This should match your application's domain.
- `challengeLifetime`: Optional. Maximum time after which a challenge must have been solved. Default is 5 minutes.

Client-side implementation

INFO

Flutter UI components for passkeys are not yet available. You'll need to implement the client-side passkey flow using the WebAuthn API directly or use a Flutter package that supports WebAuthn/FIDO2.

Using the generated endpoints

The generated client endpoints provide the following methods:

- `createChallenge()`: Creates a new challenge for registration or login.
- `register()`: Registers a new passkey for the authenticated user.
- `login()`: Authenticates a user using their passkey.